Computational Astrophysics 2023/2024
Physics of Data

Part-1: *N*-body simulations
Submission deadline: 23/12/2023

Assignment 2: May the Force….

"Don't underestimate the Force." - Darth Vader

Learning aims: Through this exercise, you will implement in the *Fireworks* Python module the methods to estimate the mutual gravitational forces acting on the particles of a *N*-body simulations. These methods will be used together with *N*-body integrators to evolve *N*-body systems. You will also start to familiarise with the *Fireworks* package you are developing and with the *Particles* class used to store the position, velocity and mass of the particles in the simulation. Finally, through the investigation of the numerical complexity of the problem you will realise what will be the capabilities of your code (i.e. how many bodies you will be realistically able to simulate).

Note: To complete the assignment, you have to implement methods to estimate the acceleration in the *Fireworks* submodule *nbodylib.dynamics (*https://gitlab.com/iogiul/pod_compastro23/-/blob/main/Fireworks/fireworks/nbodylib/dynamics.py?ref_type=heads*,* see also the documentation at this link*).* Please follow the instructions reported in the file and follow the example of the template function *acceleration_estimate_template.* Before to start to solve the tasks, create a new branch called **assignment2_forces** and work on that branch. When, you think you are ready to update the main branch (likely when you have completed all the tasks), merge this branch with your **main branch.** Don't forget to push your updates to the remote repository.

Note-2: Use the Docker container to solve the various tasks. Python and the required packages are already installed in the container. See this guide to setup and use Docker (https://stem.elearning.unipd.it/pluginfile.php/633864/mod_resource/content/7/Docker_setup_guide.pdf).

Submission: you have to submit through moodle at least what requested in Task E. Anyway, task E cannot be completed without completing all the previous tasks.

Important-1: Before to start to work on the tasks, fetch the basic remote repository https://gitlab.com/iogiul/pod_compastro23.git. Check the log and if necessary rebase your repository to include the latest updates.

Important-2: Remember to reinstall the package (python setup.py install) before to use it in the scripts and in the tests.

Tasks:

A. **Generate initial conditions:** Implement a function ***ic_random_unifirm*** to generate random initial conditions drawing from a uniform distribution for the positions, velocities, and masses of the particles. The method has to be implemented in the submodule *fireworks.ic.py.* Implement the method following the instruction reported in the module comments and taking inspiration from the already implemented function ***ic_random_normal.*** The function should allow the user to select the number of particles to generate and the minimum and maximum boundary for the positions, velocities ,and masses. Once the method is implemented test it including a test function in the file *Fireworks.tests.test_ic.py.* The test function has to check:

1. the number of particles in output needs to be the same to the one selected as input (e.g. assert len(particles)==N)

2. the minimum and maximum values of the positions, velocities and masses are to be within the boundary selected in input (e.g. assert np.min(particles.pos)>=pos_min).

To test the implementation you have to run the command `pytest test_ic.py` inside the test folder. Once the test is successful, you can pass to the next tasks.

*(**Suggestion:** use the numpy function random_uniform to draw numbers from a uniform distribution).*

B. **Basic direct force implementation:** Implement the direct-force estimate, we have seen during the lectures, in a method called ***acceleration_direct*** in the Fireworks submodule *nbodylib.dynamics.py* Follow the instruction reported in the submodule comments and in the

documentation (at this link). (The task is considered completed after a successfully testing it, see Task D).

C. **Vectorised direct force implementation:** Implement an optimised fully vectorised version of the direct force estimate adding the method ***acceleration_direct_vectorised*** in the Fireworks submodule *nbodylib.dynamics*. **You are not allowed to use the for and/or the while statement**, you have to fully exploit the vectorisation properties of the numpy arrays (numpy documentation: https://numpy.org/doc/stable/). **Suggestion-1:** notice that if you have a 2 Nx1 numpy arrays (a and b), it possibile to perform any basic operation (-,+,*,/) considering all the possibile NxN pairs simply using a.T _operation_ b (e.g. a.T - b). This returns a NxN matrix in which each i,j element contains the operation results for the (a_i,b_j) pairs. **Suggestion-2:** you can use the @ operator to perform a classical matrix multiplication between 2D matrix. E.g. if a is a NxM matrix and b is a MxW matrix, the operation a @ b represents the algebric matrix multiplication returning a NxW array. (The task is considered completed after a successful test for the implemented function, see Task D).

D. Testing: Test the implemented method ***acceleration_direct*** and ***acceleration_direct_vectorised*** using the test suite Pytest. The tests can be found in the file *test_acceleration_estimate.py* inside the folder Fireworks/tests. You have to include the two methods in the *facc_list* list and test them with the command

```
pytest test_accelereation_estimate.py
```

E. Comparison and benchmark: You are now ready to test the performance of your direct force estimate implementations and compare them with the fast-multipole tree-based gravity solver developed by Dehnen2002 (https://arxiv.org/pdf/astro-ph/0202512.pdf). This method is implemented in the function *nbodylib.dynamics.acceleration_pyfalcon.* To solve this task you have to evaluate the computational time required to estimate the particle accelerations considering the different implemented functions and different values for the particles number *N*. So:

1. Select a grid of values for the number of particles to test.

2. For each *N,* generate random initial conditions using the function you added in fireworks in task A.

3. Then, estimate the particles acceleration using the three implemented method in the submodule *fireworks.nbodylib.dynamics: acceleration_direct, acceleration_direct_ vectorised, acceleration_pyfalcon.* For each of the functions, evaluate the time required to complete the acceleration estimate. For this purpose you can use the built-in time package, e.g.:

```
import time
t1=time.perf_counter()
#operation
t2=time.perf_counter()
dt=t2-t1 # time elapsed from t1 to t2 in
s
```

4. Collect all the (*Nparticles,time)* pairs and make a plot *Nparticles vs time* plot using three different curves for the three different methods.

5. Comment the results you obtain. Are the results in line with the theoretical expectations? Comments on what you think is the realistic maximum number of particles that can be simulated using the three different functions.

F. **(Optional) Jerk estimate:** this task is optional but it is strongly suggested since this can give you the necessary tools to use high-order integrators and optimal adaptive time-steps in the next assignments. The jerk is the derivative of the acceleration and can be estimated as:

$$\vec{j}_i \equiv \frac{\mathrm{d}\vec{a}_i}{\mathrm{d}t} = -G \sum_{j=1,\, j\neq i}^{N} m_j \left[\frac{\vec{v}_{ij}}{x_{ij}^3} - 3\frac{(\vec{x}_{ij}\cdot\vec{v}_{ij})\ \vec{x}_{ij}}{x_{ij}^5} \right]$$

Implement the method *acceleration_jerk_direct* in *fireworks.nbodylib.dynamics.py.* The function has to returns both the acceleration and the jerk for al the particles. Estimate the function performance following the procedure you used in task E.

Challenge (not mandatory): Need for Speed!

As you should have realised, the bottleneck of the *N*-body simulations is the force/acceleration estimate. Through this assignment you have also realised that the direct force estimate is very computationally expensive. I challenge you to try to speed-up as much as possibile the direct acceleration estimate. Create a new function *acceleration_optimised* in *fireworks.nbodylib.dynamics* and try everything you can thinks of (crazy ideas are welcome) to optimise the implementation. Evaluate your speed-up comparing the required computational time between the optimised function and a function implementing a simple double for loop to estimate the acceleration. Make this test using a random generation of *N=3000* particles. Send me your best result and the python script you use for the test. I will evaluate your solution and publish a ranking of the three best performances (As usual, you will not have more “exam points” for this..).